

OHY

Event Management Platform

Complete Project & Technical Documentation

Client: OHY

Developed & Prepared by: Aalpha Information System

Document Type: Project, Module & Setup Documentation

Applies To: OHY Backend API, OHY Business Portal, OHY Customer Website, OHY Super Admin Portal

Table of Contents

Table of Contents	2
1. Introduction – What is OHY?	6
1.1 In One Paragraph	6
1.2 Who Uses the Platform	6
1.3 What the Platform Does	6
1.4 How OHY Helps Day-to-Day Business	7
For an event organiser (a small event company, a band, a yoga instructor, a food-truck festival): ..	7
For a customer attending events:	7
For the OHY business itself:	7
1.5 A Real Example, Start to Finish	8
2. The Four Applications at a Glance	9
2.1 Application Map	9
2.2 How They Talk to Each Other	10
2.3 One Account, Two Roles	10
3. Module 1 – Admin Module (Backend / Core API)	11
3.1 What This Module Is	11
3.2 Feature List	11
A. Identity & Access	11
B. Event Management (used by hosts)	11
C. Shopping & Orders (used by customers)	12
D. Platform Governance (used by Super Admin)	13
E. Shared & Technical Features	13
3.3 Folder Structure	14
3.4 Database Tables	15
Accounts	15
Events	15
Tickets & Sales	15
Platform Operations	16
Master Data	16
3.5 API Standards	16
3.6 Authentication & Security	17
3.7 Payments, Fees & Payouts Explained	18
Step 1 – The host connects their bank account (one time)	18
Step 2 – A customer pays	18
Step 3 – Stripe confirms the payment	18
Step 4 – Fees are calculated	18

Step 5 – The Super Admin releases the payout	19
Order status lifecycle	19
3.8 Emails	20
3.9 File & Media Storage	20
4. Module 2 – Business Module (Event Host Portal)	21
4.1 What This Module Is	21
4.2 Feature List	21
Dashboard (/dashboard)	21
Manage Events (/dashboard/events)	21
View Event (/dashboard/events/:id)	21
Overview tab	22
Attendees tab	22
Create / Edit Event (/dashboard/events/create)	22
Attendees (/dashboard/attendees)	22
My Payouts (/dashboard/payout)	22
My Profile (/dashboard/profile)	23
Help & FAQ (/dashboard/help)	23
Forgot Password (/forgot-password)	23
Screens built but currently hidden	23
4.3 The 7-Step Event Creation Wizard	24
4.4 Folder Structure	24
5. Module 3 – Customer Module (Public Event Website)	26
5.1 What This Module Is	26
5.2 Feature List	26
Home Page (/)	26
Find Events (/find-events)	26
Event Details (/event/:id)	26
Cart (/cart)	27
Checkout (/checkout)	27
Order Result Pages	27
My Account (/my-account)	27
Wishlist (/wishlist)	28
Authentication (/auth)	28
Static Pages	28
5.3 The Ticket Buying Journey	28
5.4 Folder Structure	29
6. Module 4 – Super Admin Module (Platform Control Panel)	30
6.1 What This Module Is	30

6.2 Feature List	30
Login (/auth)	30
Dashboard (/dashboard)	30
Users (/dashboard/users)	30
Attendees (/dashboard/attendees)	30
Event Hosts (/dashboard/event-hosts)	30
Events (/dashboard/events)	31
Platform Fee Settings (/dashboard/platform-fee-settings)	31
Payouts / Settlements (/dashboard/settlements)	31
User Reports (/dashboard/user-reports)	31
Feedbacks (/dashboard/feedbacks)	32
Support Inbox (/dashboard/support-inbox)	32
CMS (/dashboard/cms)	32
Profile (/dashboard/profile)	32
Placeholder screens	32
6.3 Folder Structure	32
7. Technology Stack	34
7.1 Backend Stack	34
7.2 Frontend Stack	34
Business Portal only:	35
Customer Website only:	35
7.3 Third-Party Services	35
7.4 Why These Choices	36
8. Project Setup Guide	37
8.1 Software You Need to Install First	37
8.2 Recommended Folder Layout	37
8.3 Step A – Setting Up the Backend API	38
A1. Install the PHP dependencies	38
A2. Create the environment file	38
A3. Generate the application key	38
A4. Create the database	38
A5. Fill in the .env file	38
A6. Build the database tables	39
A7. Load the starter data	39
A8. Create the storage link – do not skip this	40
A9. Start the backend	40
A10. Confirm it works	40
8.4 Step B – Setting Up the Customer Website	41

8.5 Step C – Setting Up the Business Portal	41
8.6 Step D – Setting Up the Super Admin Portal	42
8.7 Step E – Connecting Stripe	42
8.8 Step F – Connecting Email (Brevo)	43
8.9 Step G – Google Maps Key	44
9. How to Run the Projects Daily	45
9.1 Ports Used	45
9.2 Start-Up Commands	45
Terminal 1 – Backend	45
Terminal 2 – Customer Website	45
Terminal 3 – Business Portal	45
Terminal 4 – Super Admin Portal	45
Optional Terminal 5 – Stripe webhook forwarding (only needed when testing payments locally) ..	45
Building for production	46
Useful backend commands	46
9.3 Default Login Accounts	46
Super Admin	46
9.4 Verifying Everything Works	47
10. Deploying to a Live Server	48
Backend	48
Frontends	48
After deploying	49
Reference environment	49
11. Common Problems & Fixes	50
12. Reference Documents Inside the Projects	52
In OHY-API/	52
In OHY-BUSINESS/	52
In OHY-SUPER-ADMIN/	52
13. Glossary	54
14. Closing Notes	55
What you now have	55
The five things worth remembering	55
Suggested first week for a new developer	55
Recommended immediate actions on taking over	55

1. Introduction – What is OHY?

1.1 In One Paragraph

OHY is an online platform for events. People and companies who organise events – concerts, workshops, food festivals, tech meetups, art walks, conferences – use OHY to publish their event online and sell tickets for it. Ordinary customers visit the OHY website, browse events happening near them, buy tickets with a card, and receive their tickets by email. The money is collected safely by the platform, a small platform fee is kept, and the rest is paid out to the event organiser.

Think of it as a complete "event shop" – one side builds the shop, the other side buys from it, and a control room in the middle keeps everything running smoothly.

1.2 Who Uses the Platform

There are three kinds of people who use OHY, and each of them gets their own dedicated screen:

Who	What they are called in the system	What they do
A company or an individual who organises events	Event Host (also called <i>Business</i>)	Creates events, sets ticket prices, tracks sales, receives money
A person who wants to attend an event	Customer (also called <i>End User</i>)	Searches events, adds tickets to cart, pays, gets tickets
The OHY team who owns and runs the platform	Super Admin	Watches over all events, hosts and customers; sets fees; releases payouts; handles complaints

1.3 What the Platform Does

Broadly, the platform covers six areas:

1. **Event publishing** – a guided, step-by-step form where a host enters event details, uploads photos and videos, adds the venue on a map, lists the performing artists, writes terms and conditions, and creates discount coupons.
2. **Ticket selling** – several ticket types per event (for example *Early Bird*, *Regular*, *VIP*), each with its own price, stock quantity and a limit on how many one person can buy.
3. **Online payment** – customers pay by card through **Stripe**. Card details never touch the OHY servers.
4. **Money management** – the platform automatically works out the platform fee and the payment gateway charge, and shows the host exactly how much they will actually receive.
5. **Payouts** – the Super Admin releases the host's earnings for an event, and Stripe transfers the money directly into the host's connected bank account.

6. **Platform governance** — the Super Admin can hide an inappropriate event, feature a good one on the homepage, block a misbehaving host, read customer complaints and feedback, and edit the website's static pages (Privacy Policy, Terms of Use, FAQ).

1.4 How OHY Helps Day-to-Day Business

For an event organiser (a small event company, a band, a yoga instructor, a food-truck festival):

- No need to build their own website or payment system. They register once and can publish an event within an hour.
- They see live sales figures on a dashboard — how many tickets sold, how much revenue earned, how much they will actually receive after fees.
- They get a full attendee list with names, emails and phone numbers — useful at the entrance gate and for future marketing.
- Discount coupons help them push sales when a show is selling slowly.
- Money arrives directly in their bank account through Stripe; there is no manual invoicing or chasing.

For a customer attending events:

- One place to discover everything happening nearby, filtered by city, date and category.
- A familiar shopping experience — cart, coupon code, secure card payment, order confirmation.
- Tickets and order history always available in their account; nothing to print or keep track of.
- A Wishlist to save events they are interested in but not ready to book yet.
- A way to report a suspicious event or give feedback directly to the OHY team.

For the OHY business itself:

- Revenue is earned automatically on every ticket sold, through a configurable platform fee.
- The fee can be set globally for everybody, or negotiated individually for a large host — all without touching code.
- Complete visibility: how many hosts, how many customers, how many events, how much revenue, in one dashboard.
- Governance tools to keep the marketplace clean and trustworthy.

1.5 A Real Example, Start to Finish

"Brooklyn Art Walk 2026"

1. Maya runs a small art collective. She registers on the OHY website, then switches to host mode and lands on the Business dashboard.
2. She clicks Create Event and fills in the 7-step wizard — title, description, dates, photos; three ticket types (Early Bird \$25, Regular \$40, VIP \$80); the venue address, which drops a pin on Google Maps; four featured artists with their Instagram links; her terms and conditions; and a coupon ARTLOVER20 giving 20% off.
3. She completes Stripe onboarding once, so OHY knows where to send her money.
4. She clicks Publish. The event is instantly live on the OHY customer website.
5. Ryan is browsing OHY for weekend plans. He filters by New York and Art, finds Brooklyn Art Walk, adds 2 Regular tickets to his cart, applies ARTLOVER20, and pays by card.
6. Stripe confirms the payment. OHY immediately reduces the available ticket stock, generates the order, and emails Ryan his ticket confirmation.
7. Maya opens her dashboard and sees the sale appear, along with her running revenue total and the fees deducted. She can see Ryan in her attendee list.
8. After the event, the OHY Super Admin opens the Payouts screen, reviews the settlement breakdown for the event, and clicks Settle. Stripe transfers Maya's share into her bank account, and the orders are marked as settled.

That single story touches almost every feature in the platform.

2. The Four Applications at a Glance

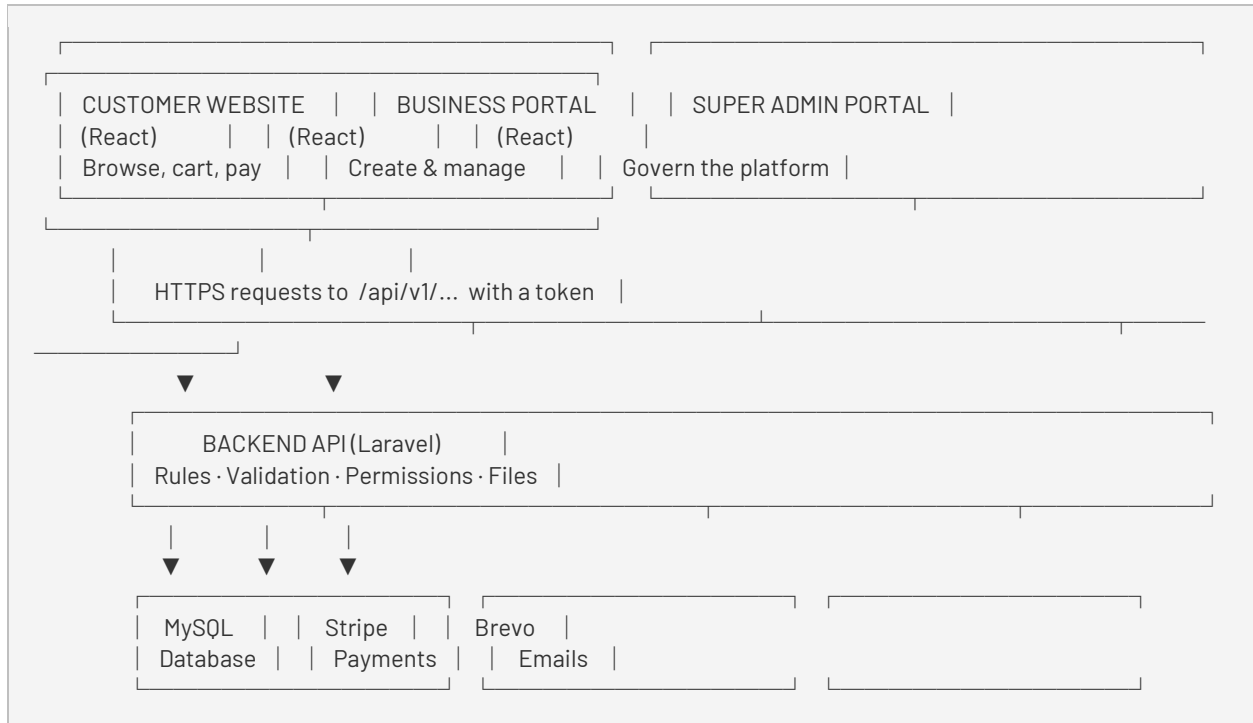
2.1 Application Map

The platform is made up of **four separate projects**. They are developed, deployed and started independently.

#	Project	Folder Name	What It Is	Technology
1	Backend API	OHY-API	The engine. Holds all data and all business rules. Has no screens of its own.	Laravel 12 (PHP) + MySQL
2	Business Portal	OHY-BUSINESS	The dashboard event hosts log into.	React 18 + Vite + TypeScript
3	Customer Website	OHY-CUSTOMER	The public website where tickets are bought.	React 18 + Vite + TypeScript
4	Super Admin Portal	OHY-SUPER-ADMIN	The internal control panel for the OHY team.	React 18 + Vite + TypeScript

Important to understand: the three React projects contain no business logic and no database access. They only display screens and call the Backend API. Every rule — pricing, fees, ticket availability, permissions — lives in the Backend API. This means a change to a business rule is made in one place only.

2.2 How They Talk to Each Other



Every request from a React app carries a **token** that identifies who is asking. The Backend checks that token and decides whether the person is allowed to do what they are asking for.

2.3 One Account, Two Roles

This is a design detail worth knowing early, because it surprises people otherwise.

- Registration and login for both customers and hosts happen on the Customer Website. The Business Portal has no sign-up page of its own – opening its home page simply redirects to the Customer Website.
- Once logged in as a customer, the user sees a "Switch to host mode" option in the header menu.
- Choosing it calls the API, which either finds the matching host account or creates one automatically using the same email, and returns a new host token.
- The browser is then sent to the Business Portal with that token in the URL. The Business Portal saves the token and the user is inside their host dashboard.

The practical effect: **one email address, one password, two dashboards**. Someone can buy a ticket in the morning and manage their own event in the afternoon without registering twice.

The Super Admin is completely separate – it has its own login screen, its own database table, and no connection to customer or host accounts.

3. Module 1 – Admin Module (Backend / Core API)

3.1 What This Module Is

This is the **central system** of the whole platform — the part that everything else depends on. It is a Laravel application that exposes a REST API. It stores every record, enforces every rule, talks to Stripe and to the email service, and keeps every uploaded image and video.

It has **no user interface**. You never "use" it directly; the three portals use it on your behalf.

Location: OHY-API (in development, this sits at `d:\wamp\www\ohy-api`)

3.2 Feature List

A. Identity & Access

Feature	Simple Explanation
Three separate account systems	Customers, Event Hosts and Super Admins are stored in three different tables. A customer can never accidentally get host powers.
Registration with email OTP	When someone registers, a 6-digit code is emailed to them. They must enter it to prove the email is real. This applies to both customers and hosts.
Login with token	On successful login the API issues a long random token. The browser stores it and sends it with every later request.
Forgot password with OTP	Three-step flow: request a code → verify the code → set a new password. Available for both customers and hosts.
Profile switching	Converts a customer session into a host session (or the reverse), creating the missing account automatically if needed.
Role checks on every route	Middleware guards make sure host routes can only be called by hosts, admin routes only by admins, and so on.

B. Event Management (used by hosts)

Feature	Simple Explanation
7-step event creation	Each step is a separate save endpoint, so a half-finished event is never lost.
Draft & publish states	An event stays private until the host clicks Publish. Published events remain fully editable.
Media handling	Thumbnail, banner, flyers and videos are uploaded, validated and stored on the server.
Ticket types & categories	Multiple ticket products per event, each with price, total stock, and a per-customer purchase limit.

Feature	Simple Explanation
Venue with map coordinates	Address plus latitude/longitude so the venue can be pinned on a map for customers.
Artists / speakers	Each with a photo and links to their social media profiles.
Terms & conditions	Stored as rich text so formatting (bold, bullet points) is preserved exactly.
Discount coupons	Percentage or flat-amount discounts, with a maximum cap, a usage limit and a validity window.
Event summary & data score	Before publishing, the host sees everything they entered plus a completeness percentage.
Attendee lists	Per-event and platform-wide lists of everyone who bought a ticket, searchable and paginated.
Host dashboard figures	Total revenue, Stripe charges deducted, platform fee deducted, and net profit.

C. Shopping & Orders (used by customers)

Feature	Simple Explanation
Public event browsing	Event lists and event detail pages that work without logging in, with search, category, city and date filters.
Shopping cart	Add, update quantity, and empty. Stock and per-user limits are re-checked on every change.
Coupon application	Only one coupon per order. The API validates the code, dates, usage count and cap before applying it.
Checkout summary	Returns the exact line-by-line breakdown the customer will pay.
Stripe Checkout session	Creates a secure Stripe-hosted payment page and returns its URL.
Order creation on payment confirmation	The order is only finalised when Stripe confirms the payment via webhook — never on a browser redirect alone.
Order history & details	Past orders with their tickets, prices and event information.
Wishlist	Save events for later; add, list and remove.
Report an event or business	Customers can flag content that looks wrong, which reaches the Super Admin.

D. Platform Governance (used by Super Admin)

Feature	Simple Explanation
Platform-wide dashboard & analytics	Counts and revenue figures across all hosts, events, customers and orders.
Master lists	Every registered customer, host, event and attendee, with filters and pagination.
Hide / unhide an event	Instantly removes an event from the public website without deleting it.
Feature / unfeature an event	Promotes an event to the highlighted section of the customer homepage.
Block / unblock a host	Stops a host from logging in or selling.
Platform fee control	One global fee for everybody, plus optional per-host overrides. Fee can be a flat amount per ticket or a percentage of the sale.
Settlements & payouts	View how much is owed for an event, see the full breakdown, and release the payment to the host through Stripe.
Reports inbox	Customer complaints with a status workflow: pending → in review → resolved.
Feedback inbox	Ratings and comments submitted by customers and hosts.
Support inbox	Support requests raised from either portal.
CMS pages	Create and edit the static pages shown on the customer website footer (Privacy Policy, Terms of Use, FAQ, and any new page).

E. Shared & Technical Features

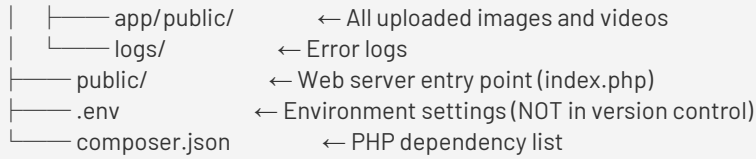
Feature	Simple Explanation
Master data endpoints	Countries, states, event categories and business categories – served to all portals so dropdowns stay consistent.
Health check endpoint	/api/v1/health confirms the API is up and the database is reachable. Useful for uptime monitoring.
Swagger API documentation	Auto-generated, browsable API reference at /api/documentation .
Consistent response format	Every endpoint returns the same JSON shape, so frontend error handling is written once.
Database transactions	Any operation that writes to several tables is wrapped in a transaction – it either fully succeeds or fully rolls back.
Detailed logging	Failures are logged with context to storage/logs/laravel.log .

3.3 Folder Structure

```

OHY-API/
├── app/
│   ├── Http/
│   │   ├── Controllers/      ← All API endpoint logic
│   │   │   ├── Host/        ← Event host endpoints
│   │   │   │   ├── EventController.php    (event creation wizard, listings, attendees)
│   │   │   │   └── HostUserController.php (host auth, profile, Stripe onboarding, dashboard)
│   │   │   ├── User/        ← Customer endpoints
│   │   │   │   ├── CartController.php     (cart, coupons, checkout summary)
│   │   │   │   ├── EventController.php    (event categories)
│   │   │   │   ├── OrderController.php    (checkout session, orders)
│   │   │   │   ├── ReportController.php   (report an event/business)
│   │   │   │   ├── UserController.php    (customer auth & profile)
│   │   │   │   ├── UserEventController.php (public event browsing)
│   │   │   │   └── WishlistController.php (wishlist)
│   │   │   ├── SuperAdmin/ ← Super Admin endpoints
│   │   │   │   ├── PlatformFeeController.php (global & per-host fees)
│   │   │   │   ├── SettlementController.php (payouts to hosts)
│   │   │   │   └── SuperAdminController.php (dashboard, lists, CMS, moderation)
│   │   │   ├── FeedbackController.php    (feedback submission)
│   │   │   ├── HealthController.php      (health check)
│   │   │   ├── MasterDataController.php  (countries, states, CMS pages)
│   │   │   ├── ProfileSwitchController.php (customer ⇄ host switching)
│   │   │   ├── StripeWebhookController.php (receives Stripe events)
│   │   │   └── SupportRequestController.php (support requests)
│   │   └── Middleware/    ← Access guards
│   │       ├── AuthenticateApiToken.php (is the token valid?)
│   │       ├── AuthenticateUser.php     (is this a customer?)
│   │       ├── AuthenticateHostUser.php (is this a host?)
│   │       ├── AuthenticateSuperAdmin.php (is this an admin?)
│   │       └── AuthenticateUserOrHost.php (either is fine)
│   ├── Models/          ← One file per database table (28 models)
│   ├── Services/        ← Reusable business logic
│   │   ├── BrevoEmailService.php (sends all emails)
│   │   ├── PlatformFeeService.php (resolves and calculates the fee)
│   │   └── StripeService.php      (all Stripe operations)
│   ├── Mail/           ← Email templates
│   │   ├── OtpEmail.php
│   │   ├── RegistrationOtpEmail.php
│   │   └── TicketPurchaseConfirmationEmail.php
│   ├── Documentation/Swagger/ ← API documentation annotations
│   └── Providers/
├── config/             ← Settings files
│   ├── constants.php    (frontend URLs used for redirects)
│   ├── services.php      (Stripe & Brevo credentials)
│   └── l5-swagger.php    (API docs settings)
├── database/
│   ├── migrations/      ← Scripts that build the database tables
│   └── seeders/         ← Scripts that insert starter/demo data
├── routes/
│   └── api.php           ← THE MASTER LIST of every API endpoint
└── storage/

```



The single most useful file for a new developer is `routes/api.php`. It lists every endpoint, grouped by who is allowed to call it, and points to the controller method that handles it. Start there.

3.4 Database Tables

The database contains **28 tables**, grouped by purpose.

Accounts

Table	Holds
<code>users</code>	Customer accounts
<code>host_users</code>	Event host accounts
<code>businesses</code>	Business details linked to a host
<code>super_admins</code>	Platform administrator accounts
<code>personal_access_tokens</code>	Active login tokens for all account types

Events

Table	Holds
<code>events</code>	The main event record
<code>event_media</code>	Thumbnails, banners, flyers, videos
<code>event_social_media</code>	Event social links
<code>venues</code>	Address and map coordinates
<code>event_artists</code>	Performers / speakers
<code>artist_social_media</code>	Artist social links
<code>event_terms_conditions</code>	Terms text

Tickets & Sales

Table	Holds
<code>ticket_categories</code>	Named groups such as Early Bird, VIP
<code>tickets</code>	Actual sellable ticket products with price and stock

Table	Holds
<code>coupons</code>	Discount codes
<code>carts</code>	Items currently in a customer's cart
<code>orders</code>	Completed purchases
<code>order_tickets</code>	Individual tickets inside an order
<code>wishlists</code>	Saved events

Platform Operations

Table	Holds
<code>platform_fees</code>	Global fee plus per-host overrides
<code>cms_pages</code>	Editable static pages
<code>user_reports</code>	Complaints from customers
<code>user_feedbacks</code>	Feedback and ratings
<code>support_requests</code>	Support enquiries

Master Data

Table	Holds
<code>countries</code>	Country list
<code>states</code>	State / province list
<code>event_categories</code>	Music, Sports, Technology, etc.
<code>business_intersections</code>	Business classification (e.g. Woman-owned)

3.5 API Standards

Every endpoint follows the same conventions, which makes the API predictable.

Base URL: `{server}/api/v1/`

Every successful response looks like this:

```
{
  "success": true,
  "data": {}
}
```

Every failed response looks like this:

```
{
  "success": false,
  "error": {
    "error_code": "E001",
    "error_message": "A clear, human-readable message"
  }
}
```

Endpoint naming uses lowercase words joined by underscores, describing the action: `get_events_list`, `create_checkout_session`, `toggle_event_hide_status`, `update_global_platform_fee`.

Common error codes:

Code	Meaning
T001	No token was sent
T002	The token is invalid
T003	The token has expired
E001	Validation failed (a field is missing or wrong)
E002	Something went wrong on the server
E003	You are not authorised to do this

HTTP status codes follow standard practice: `200` success, `401` not authenticated, `403` not allowed, `404` not found, `422` validation error, `500` server error.

3.6 Authentication & Security

- Authentication uses **Laravel Sanctum** personal access tokens.
- The token may be sent in either header: `Authorization: Bearer {token}` or `token: {token}`
- Passwords are hashed with bcrypt and are never stored or returned in readable form.
- Card details are **never** received or stored by OHY — payment happens entirely on Stripe's own hosted page.
- Email addresses must be verified with an OTP before an account becomes usable.
- Every write operation is validated on the server before it touches the database, regardless of what the frontend already checked.
- The Stripe webhook endpoint verifies a cryptographic signature so that only genuine Stripe messages are accepted.

3.7 Payments, Fees & Payouts Explained

This is the most business-critical flow in the platform, so it is worth reading carefully.

Step 1 – The host connects their bank account (one time)

Before a host can be paid, they complete **Stripe Connect Express onboarding**:

1. In the Business Portal the host clicks to create their Stripe account.
2. The API creates an Express connected account at Stripe and stores its ID against the host.
3. The API generates a secure onboarding link, and the host is sent to Stripe to enter their identity and bank details.
4. Stripe verifies them (KYC). Once complete, the host is eligible to receive money.

OHY never sees or stores the host's bank details – Stripe holds them.

Step 2 – A customer pays

1. The customer's cart is converted into a Stripe Checkout Session.
2. The customer is redirected to Stripe's hosted payment page and pays there.
3. The order is created in the database with the status `pending_payment`.

Step 3 – Stripe confirms the payment

Stripe sends a **webhook** to `/api/v1/stripe/webhook`. On receiving a confirmed payment the API:

- marks the order `paid`,
- reduces the available quantity on each ticket sold,
- increments the coupon usage count if one was applied,
- empties the customer's cart,
- records the actual Stripe processing fee against the order,
- emails the customer their ticket confirmation.

Doing this on the webhook rather than the browser redirect is deliberate. If the customer closes the browser right after paying, the order is still completed correctly.

Step 4 – Fees are calculated

Two amounts are deducted from a sale:

Deduction	Who takes it	How it is set
Stripe processing fee	Stripe	Fixed by Stripe's pricing; read back from Stripe per transaction
Platform fee	OHY	Configured by the Super Admin

The **platform fee** is resolved in this order:

1. Is there a fee configured specifically for this host? → use it.
2. Otherwise, is there a global fee configured? → use it.
3. Otherwise → no platform fee.

The fee itself can be either:

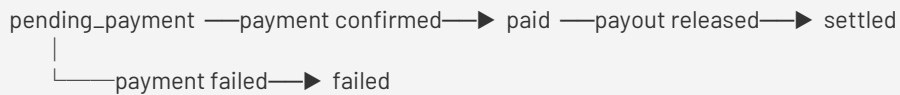
- **Flat rate** — a fixed amount per ticket (fee × number of tickets), or
- **Percentage** — a percentage of the order subtotal.

Both the host and the Super Admin can see this breakdown clearly on their dashboards.

Step 5 – The Super Admin releases the payout

1. The Super Admin opens Payouts and selects an event.
2. The system shows the settlement summary — total collected, Stripe fees, platform fee, and the net amount payable to the host.
3. The Super Admin clicks Settle.
4. A Stripe Transfer is created to the host's connected account.
5. All the orders included in that payout are marked settled with a timestamp, so nothing can be paid twice.

Order status lifecycle



3.8 Emails

All outgoing email is sent through **Brevo's HTTP API** rather than SMTP. This was a deliberate choice — many shared hosting providers block outbound SMTP ports, whereas the HTTPS API always works.

Three emails are sent by the system:

Email	When it goes out
Registration OTP	When a customer or host signs up
Password reset OTP	When someone uses "Forgot Password"
Ticket purchase confirmation	When a payment is successfully confirmed

Settings live in `.env` as `BREVO_API_KEY`, `BREVO_SENDER_EMAIL` and `BREVO_SENDER_NAME`.

3.9 File & Media Storage

- Uploads are written to `storage/app/public/`.
- Running `php artisan storage:link` creates a shortcut from `public/storage` to that folder, which is what makes files reachable from a browser.
- Files are organised into folders per record, for example `host_users/{host_user_id}/` and per-event folders.
- Frontends build image URLs using their `VITE_STORAGE_BASE_URL` setting plus the path returned by the API.

If images appear broken after a fresh installation, the storage link has almost certainly not been created. This is the single most common setup mistake.

4. Module 2 – Business Module (Event Host Portal)

4.1 What This Module Is

This is the dashboard an **event organiser** works in every day. It is where events are created, sales are monitored, attendees are looked up and earnings are tracked.

Location: OHY-BUSINESS **All screens live under:** </dashboard>

Remember from section 2.3: hosts do not register here. They register on the Customer Website and switch to host mode. Opening the Business Portal's home page redirects them there automatically.

4.2 Feature List

Dashboard (</dashboard>)

The landing screen after login. It shows:

- **Total Revenue** – everything earned from ticket sales.
- **Stripe Charges Deducted** – what the payment gateway took.
- **Platform Fee Deducted** – what OHY took.
- **Total Profit** – what the host actually receives.
- A **recent events overview** with each event's status and performance.
- A **quick action panel** with a shortcut to create a new event.

Manage Events (</dashboard/events>)

The complete list of the host's events.

- **Search** across event titles and descriptions.
- **Grid or list view** toggle.
- **Status tabs with counts:** Live / Ongoing, Upcoming, Completed, Drafts.
- Each event card shows title, status badge, date and time, venue, attendee count, ticket price, and revenue earned.
- **Row actions:** View Details, Edit Event, View Attendees, Cancel Event.
- Paginated listing.

View Event (</dashboard/events/:id>)

A deep view of one event, with four summary cards at the top (Registered, Revenue, Ticket Types, Active Coupons) and two tabs:

Overview tab

- Full description and key highlights with formatting preserved.
- All uploaded media — thumbnail, banner, flyers, videos.
- **Tickets & pricing** — for each ticket type: sold count against total, a progress bar, price, and revenue earned. Total revenue shown at the bottom.
- **Featured artists** with photos and social links.
- Terms and conditions.
- **Sidebar**: start and end date/time, venue details with a Get Directions link, event social media links, and active coupons showing code, discount, usage count and expiry.

Attendees tab

- Searchable table by name and email.
- Columns: Name (with ticket type badge), Email, Contact, Total Transactions, Total Spend.
- Summary cards: total revenue from attendees, average spend, total transactions.
- Paginated.

Create / Edit Event (/dashboard/events/create)

The 7-step wizard — described in detail in section 4.3.

Attendees (/dashboard/attendees)

A single view of **every attendee across every event** the host has run.

- Full-text search on name, email and contact.
- Date range filter on last transaction date.
- Filter by event.
- Sort by last transaction date, total spend, or number of transactions.
- Columns: Name, Email, Contact, Total Transactions, Total Spend, Last Transaction, Events Attended.
- Paginated, with the total count displayed.

My Payouts (/dashboard/payout)

Where the host tracks their money.

- Summary cards: Total Revenue, Stripe Charges Deducted, Platform Fee Deducted, Net Payable.
- A list of payouts per event, with status.

- A **breakdown dialog** that opens the full calculation for any event so the host can see exactly how the figure was reached.
- **Stripe onboarding** — if the host has not yet connected a bank account, this screen guides them through creating their Stripe account and completing verification.

My Profile (/dashboard/profile)

Three tabs:

Tab	Contents
Personal Information	Profile picture (with built-in image cropper), first and last name, email, phone, website, location (country / state / city / zip code)
Business	Business name, business type, industry, company size, tax ID, business classification, and the full business address
Security	Change password — current password, new password, confirm new password

Help & FAQ (/dashboard/help)

- Frequently asked questions for hosts.
- **Contact Support** dialog — raises a support request that lands in the Super Admin's Support Inbox.
- **Feedback** dialog — submits a rating and comment that reaches the Super Admin's Feedback screen.

Forgot Password (/forgot-password)

The three-step OTP reset flow: request the code, verify the code, set a new password.

Screens built but currently hidden

These pages exist in the codebase and work, but their menu entries are commented out in [src/components/layout/DashboardLayout.tsx](#). They can be re-enabled by uncommenting a few lines when the business decides to launch them.

Screen	Purpose
Advanced Analytics	Deeper charts on sales and attendance trends
Marketing	Promotional tools for hosts
Platform Fee	Read-only view of the fee applied to this host
Subscriptions	Planned host subscription plans

This is worth flagging to anyone taking over the project: the work is already done, it is simply not exposed in the navigation yet.

4.3 The 7-Step Event Creation Wizard

Each step saves independently, so a host can stop at any point and come back later. Unfinished events sit in the **Drafts** tab.

Step	Screen	What the host enters
1	Event Details	Title, rich-text description, category, start and end dates and times, thumbnail, banner, flyers, videos, social media links, and key highlights. Images can be cropped in the browser before upload.
2	Ticketing	Create ticket categories (Early Bird, Regular, VIP...), then for each ticket: type, price including taxes, total quantity available, ticket information, and maximum tickets per customer.
3	Venue Details	Venue name and full address with Google Places autocomplete, city, state, postal code, country, additional details, maximum attendees, a map preview to confirm the pin, and an optional venue image.
4	Event Members	Add artists or speakers one by one — photo, name, and links to Facebook, Instagram, TikTok, LinkedIn, Snapchat, X, YouTube and Spotify.
5	Terms & Conditions	A rich-text editor. Formatting is preserved exactly as customers will see it.
6	Create Coupons	Coupon code (typed or auto-generated), discount type (percentage or flat), discount value, maximum discount cap, usage limit, and validity dates.
7	Summary	Everything reviewed on one screen, with a completeness score. Then Save as Draft, Preview, or Publish Event.

Publishing makes the event immediately visible on the Customer Website. Published events stay fully editable.

4.4 Folder Structure

```

OHY-BUSINESS/
├── src/
│   ├── api/
│   │   └── services/    ← One file per feature area, each calling the backend
│   │       ├── apiClient.ts    (shared Axios setup: base URL, token, error handling)
│   │       ├── authService.ts
│   │       ├── eventService.ts
│   │       ├── attendeeService.ts
│   │       ├── dashboardService.ts
│   │       ├── payoutService.ts
│   │       ├── platformFeeService.ts
│   │       ├── profileService.ts
│   │       ├── stripeService.ts
│   │       └── feedbackService.ts

```

```

| | | └── supportService.ts
| | | └── types/      ← TypeScript definitions of API request/response shapes
| | └── components/
| |   ├── auth/      ← Login, Register and Forgot Password forms
| |   ├── events/    ← All 7 wizard steps + image croppers, Google Places, map preview
| |   ├── layout/    ← DashboardLayout (sidebar and header – edit the menu here)
| |   ├── payouts/   ← Payout breakdown dialog
| |   ├── platform-fee/ ← Platform fee dialog
| |   ├── profile/   ← Country picker, image cropper
| |   ├── stripe/    ← Stripe account creation and onboarding modals
| |   ├── support/   ← Contact support dialog
| |   ├── feedback/  ← Feedback dialog
| |   └── ui/        ← shadcn/ui base components (buttons, dialogs, tables...)
| ├── pages/        ← One file per screen
| ├── hooks/        ← Reusable logic (events list, attendees list, debounce)
| ├── store/auth-store/ ← Logged-in user state (Zustand)
| ├── config/       ← Google Maps configuration
| ├── utils/        ← Helper functions
| ├── App.tsx       ← ROUTE DEFINITIONS – every screen URL is listed here
| └── main.tsx      ← Application entry point
└── public/        ← Logos and static images
└── .env           ← API and storage URLs
└── package.json   ← Dependency list and npm scripts
└── tailwind.config.ts ← Colours, fonts and design tokens
└── vite.config.ts ← Dev server port and path aliases

```

5. Module 3 – Customer Module (Public Event Website)

5.1 What This Module Is

This is the **public face of OHY** – the website anyone can visit. It is where events are discovered and tickets are bought, and it is also the single registration and login point for the whole platform.

Location: OHY-CUSTOMER

Most of it works **without logging in**. An account is only required at the point of adding to cart and paying.

5.2 Feature List

Home Page (/)

- Hero section with a search bar and an animated headline.
- **Featured Events** – the events the Super Admin has chosen to promote.
- **Events Near Me** – events filtered by the visitor's location.
- "Why choose OHY" and platform features sections.
- A **Create Event** call-to-action inviting visitors to become hosts.
- Footer with links to the CMS-managed pages (Privacy Policy, Terms of Use, FAQ).

Find Events (/find-events)

The main discovery screen.

- Free-text **search**.
- Filters for **category**, **location / city** and **date**.
- **Clear filters** button.
- Paginated results.
- A **wishlist heart** on every event card for one-click saving.

Event Details (/event/:id)

Everything a customer needs to decide.

- Image carousel of the event's photos.
- Full description and key highlights with original formatting.
- Date, time and full venue address, with a map location.
- **Ticket selection** – all available ticket types with prices and remaining availability; sold-out types are clearly marked.

- Featured artists with their social links.
- Terms and conditions.
- Event social media links.
- **Add to cart**, **Add to wishlist**, and **Share**.
- **Report this event** for anything that looks inappropriate.

Cart (/cart)

- All selected tickets grouped by event.
- Increase or decrease quantities, or remove an item — with live re-checking of stock and per-customer limits.
- **Apply a coupon code**, or pick from the list of coupons available for the events in the cart.
- Order summary with subtotal, discount, fees and the final total.
- Proceed to checkout.

Checkout (/checkout)

- Contact information (pre-filled from the customer's profile).
- Billing address.
- Final order summary.
- **Pay** — which redirects to Stripe's secure hosted payment page.

Order Result Pages

Page	Purpose
/order/verifying	Shown immediately after returning from Stripe, while the system waits for the payment to be confirmed
/order-confirmed	Success — shows the digital ticket, order number, QR code and event details
/order-failed	Payment did not go through, with an option to try again

My Account (/my-account)

A sidebar-driven area with:

Section	Contents
Account	Personal details, profile photo, phone, address (country / state / city / zip)
Password	Change password

Section	Contents
Orders	All past orders, and a detail view for each one showing tickets, prices and event information
Help	Support request form
FAQ	Frequently asked questions

Also available from the header menu:

- **Give Feedback** — a rating and comment sent to the OHY team.
- **Switch to host mode** — the one-click route into the Business Portal described in section 2.3.

Wishlist (/wishlist)

All saved events in one place, with the option to remove an item or jump straight to the event page.

Authentication (/auth)

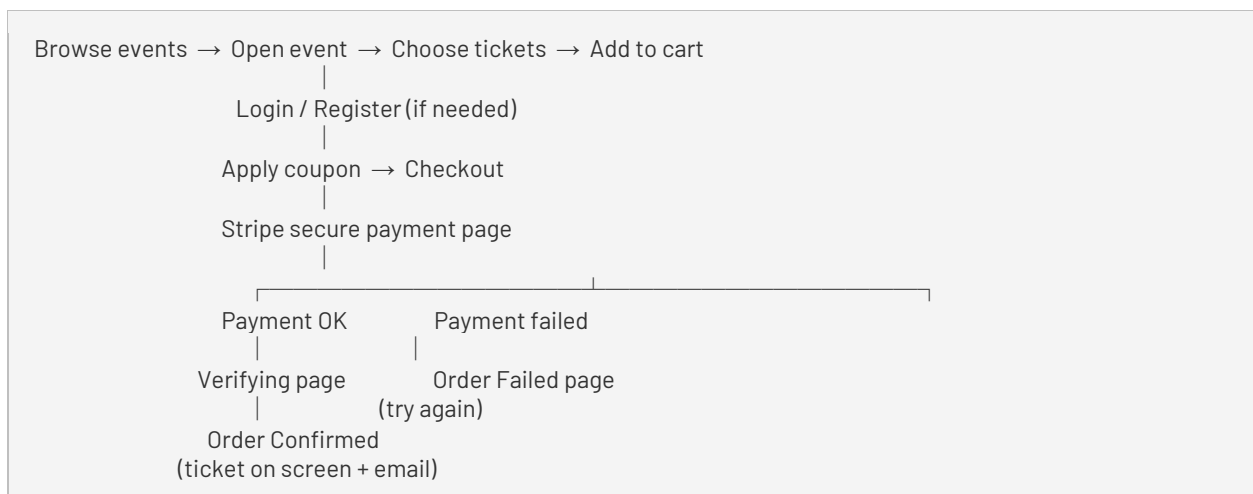
A single screen with **Login** and **Register** tabs.

- Registration collects the customer's details and sends an **email OTP** which must be entered to activate the account.
- */forgot-password* runs the three-step OTP reset flow.

Static Pages

/privacy-policy, */terms-of-use* and */faq* pull their content from the **CMS pages** the Super Admin edits. Changing this content requires no code deployment.

5.3 The Ticket Buying Journey



5.4 Folder Structure

```

OHY-CUSTOMER/
├── src/
│   ├── api/
│   │   ├── client.ts      ← Axios instance: base URL, token header, error handling
│   │   ├── endpoints.ts   ← Every backend URL in one place
│   │   ├── errors.ts      ← Error message mapping
│   │   ├── storage.ts     ← Reads and writes the token in localStorage
│   │   ├── types.ts      ← TypeScript definitions
│   │   ├── services/      ← auth, events, cart, orders, wishlist, user, reports,
│   │   │   │               feedback, support
│   │   └── hooks/        ← React Query hooks for events and profile
│   ├── components/
│   │   ├── Header.tsx     ← Navigation, account menu, "Switch to host mode"
│   │   ├── Footer.tsx    ← CMS page links
│   │   ├── HeroSection.tsx, FeaturedEvents.tsx, EventsNearMe.tsx,
│   │   │   WhyChooseSection.tsx, FeaturesSection.tsx, CreateEventSection.tsx
│   │   ├── ImageCarousel.tsx, MarqueeRibbon.tsx, TypewriterText.tsx, MobileMenu.tsx
│   │   ├── ReportDialog.tsx, FeedbackDialog.tsx
│   │   ├── auth/         ← Login and Register tabs
│   │   └── ui/           ← shadcn/ui base components
│   ├── contexts/
│   │   ├── CartContext.tsx ← Cart state shared across the whole app
│   │   └── WishlistContext.tsx ← Wishlist state shared across the whole app
│   ├── pages/            ← One file per screen
│   ├── hooks/            ← Reusable logic
│   ├── utils/            ← Helper functions
│   ├── App.tsx           ← ROUTE DEFINITIONS + the token handler used by profile switching
│   └── main.tsx          ← Application entry point
├── public/              ← Logos and sample event images
├── .env                 ← API, storage and Business Portal URLs
├── package.json
├── tailwind.config.ts
└── vite.config.ts
  
```

6. Module 4 – Super Admin Module (Platform Control Panel)

6.1 What This Module Is

This is the **internal control room** used only by the OHY team. It is not linked from anywhere on the public site and has its own separate login.

Location: OHY-SUPER-ADMIN **All screens live under:** /dashboard

6.2 Feature List

Login (/auth)

A dedicated login screen. Super Admin credentials live in their own database table and have no connection to customer or host accounts. If a session expires, the app automatically returns the admin to this screen with a clear message.

Dashboard (/dashboard)

The overview of the entire platform:

- Total events, active events and total revenue.
- Counts of registered customers, hosts and attendees.
- Charts and trends across the platform.
- Recent activity.

Users (/dashboard/users)

Every registered **customer**.

- Search and filter, including by city and zip code.
- Columns showing contact details, location and registration date.
- Paginated.

Attendees (/dashboard/attendees)

Every person who has bought a ticket anywhere on the platform, with their spend and transaction history.

Event Hosts (/dashboard/event-hosts)

Every registered **event host**.

- Search and filter.
- A **host details modal** showing their full profile, business information and activity.

- **Block / Unblock** — instantly stops or restores a host's access, with a confirmation dialog.

Events (/dashboard/events)

Every event on the platform, from every host.

- Filter by category, status, venue city and date; free-text search.
- **Hide / Unhide** — removes an event from the public website without deleting it, with a confirmation dialog.
- **Feature / Unfeature** — promotes an event to the Featured section of the customer homepage.
- **View Event** (/dashboard/events/:eventId) — the complete event record including tickets, sales, venue, artists, coupons and attendees.

Platform Fee Settings (/dashboard/platform-fee-settings)

Where the platform's commercial terms are configured.

- **Global platform fee** — applies to every host who does not have a specific arrangement. Can be set as a flat rate per ticket or a percentage of the sale.
- **Per-host fees** — override the global fee for an individual host, for negotiated deals.
- A list of every host-specific fee currently in force, each of which can be edited or removed.

Payouts / Settlements (/dashboard/settlements)

Where hosts actually get paid.

- A list of events with money waiting to be released.
- **Settlement summary** per event: total collected, Stripe charges, platform fee, net payable to the host.
- A **breakdown modal** showing the order-by-order detail behind those figures.
- **Settle** — a confirmation dialog, then a Stripe Transfer is created to the host's connected bank account and the included orders are permanently marked as settled.

User Reports (/dashboard/user-reports)

Complaints raised by customers about an event or a business.

- Full list with a detail modal showing the report and who submitted it.
- Status workflow: Pending → In Review → Resolved.

Feedbacks (/dashboard/feedbacks)

Ratings and comments submitted by customers and hosts, with a modal showing who submitted each one.

Support Inbox (/dashboard/support-inbox)

Support requests raised from the Customer Website and the Business Portal, with submitter details.

CMS (/dashboard/cms)

Content management for the customer website's static pages.

- List all pages with their status.
- **Create** a new page (/dashboard/cms/create).
- **Edit** an existing page (/dashboard/cms/edit/:id) using a rich-text editor.
- **Delete** a page, with confirmation.

Each page has a URL slug, which is how the customer website finds it. This is how Privacy Policy, Terms of Use and FAQ are maintained – no developer involvement needed.

Profile (/dashboard/profile)

The admin's own details and password.

Placeholder screens

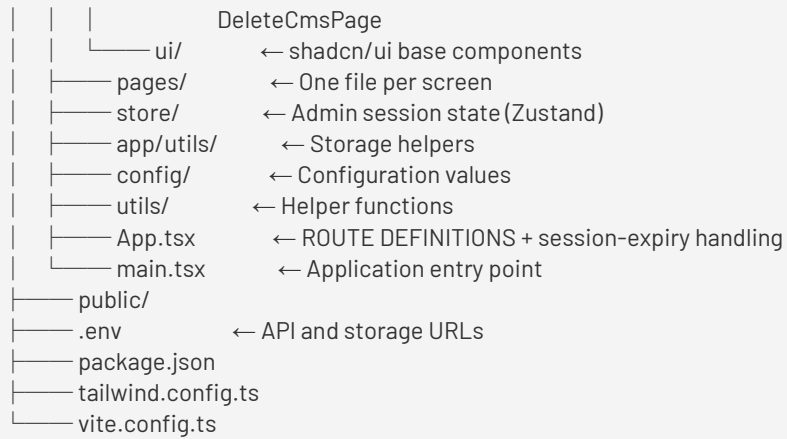
The navigation also defines **Orders**, **Analytics**, **Settings** and **Help & FAQ**. These currently render a "Coming soon" placeholder and are reserved for future phases.

6.3 Folder Structure

```

OHY-SUPER-ADMIN/
├── src/
│   ├── api/
│   │   ├── client.ts      ← Axios instance with token handling and session-expiry events
│   │   ├── endpoints.ts   ← Every backend URL in one place
│   │   ├── errors.ts, storage.ts, types.ts
│   │   ├── services/      ← auth, dashboard, events, users, reports, feedbacks,
│   │   │                   support requests, settlements, platform fees, CMS,
│   │   │                   states, event categories, venue locations,
│   │   │                   business intersections
│   │   └── hooks/         ← ~35 React Query hooks, one per API operation
│   ├── components/
│   │   ├── layout/        ← DashboardLayout (sidebar and header — edit the menu here)
│   │   └── modals/        ← BlockHost, HideEvent, HostDetails, ReportDetail,
│   │                       SubmitterDetail, SettleEventPayout,
│   │                       SettlementBreakdown, SettlementConfirmation,
│   │                       UpdateGlobalPlatformFee, UpdateHostPlatformFee,

```



7. Technology Stack

7.1 Backend Stack

Technology	Version	What it is used for
PHP	8.2 or higher	The programming language
Laravel	12.x	The web framework – routing, database access, validation, file handling
MySQL	5.7 or higher (8.0 recommended)	The database
Laravel Sanctum	4.x	Token-based API authentication
Stripe PHP SDK	19.x	Payments, Connect onboarding, transfers, webhooks
L5-Swagger (OpenAPI)	9.x	Auto-generated API documentation
Composer	2.x	PHP dependency manager
Eloquent ORM	Built into Laravel	Talking to the database using PHP objects instead of raw SQL
Carbon	Built into Laravel	Date and time handling

Development tools included: PHPUnit (testing), Laravel Pint (code formatting), Laravel Pail (live log viewing), Faker (demo data generation).

7.2 Frontend Stack

All three portals use **exactly the same stack**, which means a developer who learns one can work on all three.

Technology	Version	What it is used for
React	18.3	Building the user interface
TypeScript	5.8	Adds type safety to JavaScript, catching mistakes before they run
Vite	5.4	Development server and production build tool – very fast
Tailwind CSS	3.4	Styling using utility classes
shadcn/ui + Radix UI	latest	Ready-made accessible components – dialogs, dropdowns, tables, tabs
React Router	6.30	Page navigation

Technology	Version	What it is used for
TanStack React Query	5.83	Fetching, caching and refreshing server data
Axios	1.x	Making HTTP calls to the backend
Zustand	5.x	Lightweight state management (Business & Super Admin)
React Hook Form + Zod	7.x / 3.x	Form handling and validation
Recharts	2.15	Charts on the dashboards
Lucide React	0.462	Icon set
date-fns	3.6	Date formatting
Sonner	1.7	Toast notifications

Business Portal only:

Technology	Used for
TipTap	The rich-text editor for descriptions and terms
@react-google-maps/api + use-places-autocomplete	Venue address autocomplete and map preview
react-image-crop	Cropping images before upload
crypto-js	Client-side encryption helpers

Customer Website only:

Technology	Used for
qrcode.react	Rendering the QR code on the digital ticket
embla-carousel-react	The event image carousel

Node.js 18 or higher is required to run any of the three frontends.

7.3 Third-Party Services

Service	Purpose	What you need
Stripe	Card payments, host onboarding (Connect Express), payouts	Publishable key, secret key, webhook signing secret
Brevo	Transactional email (OTPs, ticket confirmations)	API key and a verified sender email

Service	Purpose	What you need
Google Maps Platform	Address autocomplete and map preview when creating an event	Maps JavaScript API key with the Places API enabled

7.4 Why These Choices

- **Laravel** gives a well-understood, well-documented structure. Any PHP developer can find their way around it quickly.
- **React with Vite** keeps the three portals fast to develop and fast to load, and lets the same components and patterns be reused across all of them.
- **TypeScript** catches a whole class of bugs — a wrong field name or a missing value — before the code ever runs.
- **Stripe** removes the entire burden of PCI compliance. OHY never handles card numbers.
- **Brevo over HTTP** avoids the SMTP port blocking that shared hosting providers commonly impose.
- **shadcn/ui** ships components as source code inside the project rather than as an external dependency, so they can be customised freely without fighting a library.

8. Project Setup Guide

This section takes a completely fresh machine to a fully working platform.

8.1 Software You Need to Install First

Software	Minimum Version	Where to get it	Why
PHP	8.2	Bundled with WAMP / XAMPP / Laragon, or php.net	Runs the backend
Composer	2.x	getcomposer.org	Installs backend libraries
MySQL	5.7 (8.0 preferred)	Bundled with WAMP / XAMPP	Stores all data
Node.js	18 LTS or higher	nodejs.org	Runs the three frontends
npm	Comes with Node.js	—	Installs frontend libraries
Git	any recent	git-scm.com	Version control

Required PHP extensions (usually enabled by default in WAMP/XAMPP — confirm in `php.ini`): `pdo_mysql`, `mbstring`, `openssl`, `tokenizer`, `xml`, `ctype`, `json`, `bcmath`, `fileinfo`, `curl`, `gd`

Verify your installation:

```
php -v      # should show 8.2 or higher
composer -V # should show 2.x
node -v     # should show 18 or higher
npm -v
mysql --version
```

Recommended editor: Visual Studio Code, with the extensions PHP Intelephense, ESLint, Prettier and Tailwind CSS IntelliSense.

8.2 Recommended Folder Layout

Keep all four projects side by side:

```
OHY/
├── OHY-API/      ← Backend (Laravel)
├── OHY-BUSINESS/ ← Business Portal (React)
├── OHY-CUSTOMER/ ← Customer Website (React)
└── OHY-SUPER-ADMIN/ ← Super Admin Portal (React)
```

On Windows with WAMP, place OHY-API inside C:\wamp64\www\ (or wherever your web root is) so it is served directly. The three React projects can live anywhere.

Set up the backend first. The three frontends are useless without it.

8.3 Step A – Setting Up the Backend API

A1. Install the PHP dependencies

```
cd OHY-API  
composer install
```

This reads `composer.json` and downloads everything into `vendor/`. It takes a few minutes the first time.

A2. Create the environment file

The `.env` file holds all the settings that change between machines — database passwords, API keys, URLs. It is deliberately excluded from version control.

```
cp .env.example .env
```

On Windows Command Prompt use `copy .env.example .env`.

A3. Generate the application key

```
php artisan key:generate
```

This writes a unique `APP_KEY` used for encrypting data. **Never change it after the system holds live data.**

A4. Create the database

Create an empty MySQL database — for example in phpMyAdmin, or from the command line:

```
CREATE DATABASE ohy_events CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

A5. Fill in the `.env` file

Open `.env` and set the following. The values in the examples are for local development.

```
# — Application —————  
APP_NAME="OHY Events"  
APP_ENV=local  
APP_DEBUG=true  
APP_URL=http://localhost:8000  
  
# — Database —————  
DB_CONNECTION=mysql  
DB_HOST=127.0.0.1  
DB_PORT=3306  
DB_DATABASE=ohy_events  
DB_USERNAME=root  
DB_PASSWORD=
```

```
# — Frontend URLs (used for redirects after payment) —
EVENT_FRONTEND_URL=http://localhost:8081 # Customer Website
HOST_FRONTEND_URL=http://localhost:8080 # Business Portal

# — Stripe (see section 8.7) —
STRIPE_KEY=pk_test_XXXXXXXXXXXX
STRIPE_SECRET=sk_test_XXXXXXXXXXXX
STRIPE_WEBHOOK_SECRET=whsec_XXXXXXXXXXXX

# — Email via Brevo (see section 8.8) —
BREVO_API_KEY=xkeysib-XXXXXXXXXXXX
BREVO_SENDER_EMAIL=no-reply@yourdomain.com
BREVO_SENDER_NAME="OHY Platform"

# — Storage & sessions —
FILESYSTEM_DISK=local
SESSION_DRIVER=database
QUEUE_CONNECTION=database
CACHE_STORE=database
LOG_CHANNEL=stack
LOG_LEVEL=debug
```

Note: the shipped .env.example is the default Laravel template and does not yet include the Stripe, Brevo and frontend-URL entries. Copy the block above into your .env so nothing is missed. It is worth updating .env.example to match, as a favour to the next person.

A6. Build the database tables

```
php artisan migrate
```

This runs every file in `database/migrations/` in order and creates all 28 tables.

A7. Load the starter data

```
php artisan db:seed
```

This inserts:

- The default Super Admin account
- The country and state lists
- Event categories
- Business classifications
- Two demo event hosts (one business account, one personal) with four events each
- Ten demo customers with orders and cart items

Skip this on a production server if you do not want demo data — but you will still need the Super Admin, countries, states and categories. Run those individually:

```
php artisan db:seed --class=SuperAdminSeeder
php artisan db:seed --class=CountriesSeeder
php artisan db:seed --class=StatesTableSeeder
php artisan db:seed --class=EventCategoriesSeeder
php artisan db:seed --class=BusinessIntersectionsSeeder
```

A8. Create the storage link – do not skip this

```
php artisan storage:link
```

This makes uploaded images and videos reachable from a browser. Without it, **every image on every portal will be broken**.

A9. Start the backend

```
php artisan serve
```

The API is now running at **http://localhost:8000**.

A10. Confirm it works

Open in a browser:

- **http://localhost:8000/api/v1/health** → should return `{"success": true, "data": {"status": "healthy", ...}}`
- **http://localhost:8000/api/documentation** → the browsable Swagger API reference

If the health check returns healthy, the backend and the database are correctly connected.

8.4 Step B – Setting Up the Customer Website

Set this up second, because it is where all registration and login happens.

```
cd OHY-CUSTOMER
npm install
```

Create a `.env` file in the project root:

```
VITE_API_BASE_URL=http://localhost:8000/api/v1
VITE_STORAGE_BASE_URL=http://localhost:8000/storage
VITE_HOST_FRONTEND_URL=http://localhost:8080
```

Setting	Meaning
<code>VITE_API_BASE_URL</code>	Where the backend lives
<code>VITE_STORAGE_BASE_URL</code>	Where uploaded images are served from
<code>VITE_HOST_FRONTEND_URL</code>	Where to send a user who chooses "Switch to host mode"

Start it on port **8081**:

```
npm run dev -- --port 8081
```

Open **`http://localhost:8081`**.

The Vite config defaults to port 8080 for all three frontends, so at least two of them must be given an explicit port. The convention used throughout this document is Business 8080, Customer 8081, Super Admin 8082 – matching the backend's default `EVENT_FRONTEND_URL` and `HOST_FRONTEND_URL` values.

8.5 Step C – Setting Up the Business Portal

```
cd OHY-BUSINESS
npm install
```

Create a `.env` file:

```
VITE_API_BASE_URL=http://localhost:8000/api/v1
VITE_STORAGE_BASE_URL=http://localhost:8000/storage
VITE_EVENT_FRONT_END_URL=http://localhost:8081
```

`VITE_EVENT_FRONT_END_URL` is where visitors are redirected for login and registration – it must point at the Customer Website.

Add your Google Maps key for venue address autocomplete:

```
VITE_GOOGLE_MAPS_API_KEY=your_google_maps_key
```

Please note: `src/config/googleMaps.ts` contains a hardcoded fallback API key that is used when this variable is not set. Replace it with your own key and remove the fallback — a key committed in source code can be copied and used by anyone.

Start it on port **8080** (its default):

```
npm run dev
```

Open **`http://localhost:8080`** — it will redirect you to the Customer Website to log in, which is the expected behaviour.

8.6 Step D — Setting Up the Super Admin Portal

```
cd OHY-SUPER-ADMIN
npm install
```

Create a **.env** file:

```
VITE_API_BASE_URL=http://localhost:8000/api/v1
VITE_STORAGE_BASE_URL=http://localhost:8000/storage
```

Start it on port **8082**:

```
npm run dev -- --port 8082
```

Open **`http://localhost:8082`** and log in with the seeded Super Admin account (see section 9.3).

8.7 Step E — Connecting Stripe

Payments will not work until this is done. Use **test mode** for development.

1. Create a free account at stripe.com and stay in Test mode.
2. Go to Developers → API keys and copy: the Publishable key (`pk_test_...`) → `STRIPE_KEY`; the Secret key (`sk_test_...`) → `STRIPE_SECRET`
3. Enable Stripe Connect: go to Connect → Get started and choose the Express account type. This is what lets hosts receive payouts.
4. Set up the webhook. The backend must be told when a payment succeeds.

For local development, install the Stripe CLI and forward events to your machine:

```
stripe login
stripe listen --forward-to http://localhost:8000/api/v1/stripe/webhook
```

The CLI prints a signing secret (`whsec_...`) — put it in `STRIPE_WEBHOOK_SECRET`.

For a live or staging server, go to Developers → Webhooks → Add endpoint and use:

```
https://your-api-domain.com/api/v1/stripe/webhook
```

Subscribe to at least these events:

- `checkout.session.completed`
- `checkout.session.async_payment_succeeded`
- `checkout.session.async_payment_failed`
- `payment_intent.payment_failed`
- `account.updated`
- `transfer.created`
- `transfer.paid`

Copy the endpoint's signing secret into `STRIPE_WEBHOOK_SECRET`.

Clear the config cache so the new values are picked up:

```
php artisan config:clear
```

Test card numbers (test mode only):

Card Number	Result
4242 4242 4242 4242	Payment succeeds
4000 0000 0000 0002	Payment declined

Use any future expiry date, any 3-digit CVC, and any postal code.

8.8 Step F — Connecting Email (Brevo)

Without this, OTP emails are never delivered and nobody can complete registration.

1. Create a free account at brevo.com.
2. Go to SMTP & API → API Keys and generate a new API key (v3).
3. Put it in `.env` as `BREVO_API_KEY`.
4. Verify a sender address under Senders & IP, and put that address in `BREVO_SENDER_EMAIL`.
5. Set `BREVO_SENDER_NAME` to whatever name should appear in recipients' inboxes.
6. Run `php artisan config:clear`.

Test it by registering a new customer on the Customer Website — the OTP email should arrive within a few seconds. If it does not, check `storage/logs/laravel.log`.

8.9 Step G — Google Maps Key

Needed for the venue address autocomplete and map preview in the Business Portal's event wizard.

1. Go to the Google Cloud Console and create a project.
2. Enable Maps JavaScript API and Places API.
3. Create an API key under Credentials.
4. Restrict the key to your domains (localhost, plus your live domains) so it cannot be abused.
5. Add it to the Business Portal's `.env`.

Without a key, the rest of the event wizard still works — the host simply types the address manually instead of using autocomplete.

9. How to Run the Projects Daily

9.1 Ports Used

Application	Local URL	Port
Backend API	http://localhost:8000	8000
Business Portal	http://localhost:8080	8080
Customer Website	http://localhost:8081	8081
Super Admin Portal	http://localhost:8082	8082

These ports are not arbitrary — the backend's `config/constants.php` defaults expect the Customer Website on 8081 and the Business Portal on 8080. If you change them, update the `.env` files on both sides.

9.2 Start-Up Commands

Open **four terminal windows** and leave each running:

Terminal 1 – Backend

```
cd OHY-API
php artisan serve
```

Terminal 2 – Customer Website

```
cd OHY-CUSTOMER
npm run dev -- --port 8081
```

Terminal 3 – Business Portal

```
cd OHY-BUSINESS
npm run dev
```

Terminal 4 – Super Admin Portal

```
cd OHY-SUPER-ADMIN
npm run dev -- --port 8082
```

Optional Terminal 5 – Stripe webhook forwarding (only needed when testing payments locally)

```
stripe listen --forward-to http://localhost:8000/api/v1/stripe/webhook
```

All three React apps hot-reload: save a file and the browser updates instantly.

Building for production

```
# Each frontend
npm run build    # output goes into the dist/ folder

# Backend
php artisan config:cache
php artisan route:cache
```

Useful backend commands

Command	What it does
<code>php artisan migrate</code>	Apply any new database changes
<code>php artisan migrate:fresh --seed</code>	Wipes everything and rebuilds from scratch with demo data
<code>php artisan config:clear</code>	Reload .env after editing it
<code>php artisan cache:clear</code>	Clear the application cache
<code>php artisan storage:link</code>	Recreate the public storage link
<code>php artisan l5-swagger:generate</code>	Regenerate the API documentation
<code>php artisan test</code>	Run the test suite
<code>php artisan route:list</code>	Print every registered route

⚠ `migrate:fresh` deletes all data. Never run it on a live server.

9.3 Default Login Accounts

These come from the seeders and exist only in development.

Super Admin

Field	Value
URL	<code>http://localhost:8082</code>
Email	<code>owner@ohy.com</code>
Password	<code>SuperSecure@123</code>

Change this password immediately on any live server.

Demo hosts and customers are created by `BusinessAccountHostSeeder`, `PersonalAccountHostSeeder` and `EndUsersSeeder`. Open those files in `database/seeder/` to see their email addresses and passwords.

9.4 Verifying Everything Works

Run through this checklist after a fresh setup. If all five pass, the whole platform is working end to end.

#	Check	Expected result
1	Open <code>http://localhost:8000/api/v1/health</code>	<code>{"success": true, ... "status": "healthy"}</code>
2	Open the Customer Website and register a new account	OTP email arrives; account activates after entering it
3	Log in, open the account menu, choose Switch to host mode	You land on the Business dashboard, already logged in
4	Create and publish an event in the Business Portal	It appears on the Customer Website
5	Buy a ticket using test card <code>4242 4242 4242 4242</code>	Order Confirmed page shows; confirmation email arrives; the sale appears on the host dashboard and in the Super Admin's lists

10. Deploying to a Live Server

Backend

Step	Detail
Server requirements	PHP 8.2+, MySQL 5.7+, Apache or Nginx, Composer
Document root	Must point at the public/ folder, not the project root
Install	<code>composer install --optimize-autoloader --no-dev</code>
Environment	<code>APP_ENV=production</code> , <code>APP_DEBUG=false</code> , real database credentials, live Stripe keys
Database	<code>php artisan migrate --force</code>
Storage	<code>php artisan storage:link</code>
Caching	<code>php artisan config:cache</code> and <code>php artisan route:cache</code>
Permissions	<code>storage/</code> and <code>bootstrap/cache/</code> must be writable by the web server
SSL	Required — Stripe will not deliver webhooks to a non-HTTPS endpoint

Frontends

For each of the three React projects:

1. Create a `.env` (or `.env.production`) pointing at the live API and storage URLs.
2. Run `npm run build`.
3. Upload the contents of the generated `dist/` folder to the web root of that site's domain.
4. Configure the web server to rewrite all unmatched routes to `index.html` — these are single-page applications, and without this rule any direct link or page refresh returns a 404.

Example Apache `.htaccess` for a frontend:

```
<IfModule mod_rewrite.c>
RewriteEngine On
RewriteBase /
RewriteRule ^index\.html$ - [L]
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule . /index.html [L]
</IfModule>
```

After deploying

- Update `EVENT_FRONTEND_URL` and `HOST_FRONTEND_URL` in the backend `.env` to the live domains.
- Register the live webhook endpoint in the Stripe dashboard and update `STRIPE_WEBHOOK_SECRET`.
- Change the Super Admin password.
- Confirm the health endpoint responds on the live domain.
- Restrict the Google Maps API key to the live domains.

Reference environment

For context, the project has been running on the following staging setup:

Application	URL
Backend API	<code>https://ohy-api.aalpha.co.in/public/api/v1</code>
File storage	<code>https://ohy-api.aalpha.co.in/public/storage</code>
Customer Website	<code>https://ohy-frontend.aalpha.co.in</code>
Business Portal	<code>https://ohy-back.aalpha.co.in</code>

Note the `/public` segment in the API URL — that hosting setup serves the project root rather than the `public/` folder. On a properly configured server the `/public` segment disappears, and the frontend `.env` files must be adjusted accordingly.

11. Common Problems & Fixes

Problem	Likely Cause	Fix
All images are broken across every portal	The storage link was never created	Run <code>php artisan storage:link</code> in the backend
Every API call fails with a CORS error	The frontend origin is not allowed by the backend	The project has no published CORS config file. Create one with <code>php artisan config:publish cors</code> , add the frontend URLs to <code>allowed_origins</code> , then run <code>php artisan config:clear</code> . (On the current hosting this is handled at the web-server level.)
Changes to <code>.env</code> seem to have no effect	Laravel has cached the old configuration	Run <code>php artisan config:clear</code>
OTP emails never arrive	Brevo key missing, wrong, or sender not verified	Check <code>BREVO_API_KEY</code> and <code>BREVO_SENDER_EMAIL</code> ; read <code>storage/logs/laravel.log</code>
Payment succeeds at Stripe but the order stays "Verifying"	The webhook is not reaching the backend	Run <code>stripe listen</code> locally, or check the endpoint and signing secret in the Stripe dashboard
500 Internal Server Error on every request	Missing <code>APP_KEY</code> , or vendor/ not installed	Run <code>composer install</code> then <code>php artisan key:generate</code>
Frontend shows a blank white page	Wrong <code>VITE_API_BASE_URL</code> , or the backend is not running	Check the <code>.env</code> and the browser console; confirm the health endpoint responds
"Port 8080 already in use"	Two frontends started with the same default port	Start with <code>npm run dev -- --port 8081</code> (or 8082)
Direct links 404 after deployment	No SPA rewrite rule on the web server	Add the <code>.htaccess</code> (or Nginx <code>try_files</code>) rule from section 10
Address autocomplete does nothing in the event wizard	Google Maps key missing or Places API not enabled	Add the key and enable both Maps JavaScript API and Places API
Host cannot be paid out	Stripe Connect onboarding not finished	The host must complete onboarding from the Payout screen in the Business Portal
Class not found after pulling new code	Autoloader out of date	Run <code>composer dump-autoload</code>

Where to look when something breaks:

Location	What it tells you
<code>OHY-API/storage/logs/laravel.log</code>	Backend errors with full context
Browser Console tab	Frontend JavaScript errors

Location	What it tells you
Browser Network tab	The exact request sent and response received
Stripe Dashboard → Developers → Events	Whether Stripe sent the webhook and whether it was accepted
Brevo Dashboard → Logs	Whether an email was actually delivered

12. Reference Documents Inside the Projects

Alongside this document, each project carries its own detailed reference material. These are useful when working on a specific area.

In OHY-API/

File	Contents
README.md	Quick start guide
PRD.md	Full product requirements for the Business and Customer modules — every screen and field described
API_PRD.md	API development standards: response formats, validation patterns, controller structure, commenting rules
DATABASE_PRD.md	Every table, every column, with types and relationships
CURSOR_RULES.md	Coding conventions the project follows
docs/PRD_SuperAdmin.md	Super Admin module requirements
SUPER_ADMIN_FRONTEND_IMPLEMENTATION_GUIDE.md	Frontend implementation notes for the admin portal
SUPER_ADMIN_CMS_FRONTEND_IMPLEMENTATION_GUIDE.md	CMS feature implementation notes
OHY-DB.sql	A full database dump — an alternative to running migrations
.cursor/rules/	Machine-readable coding rules

In OHY-BUSINESS/

File	Contents
OHY-Project-Understanding-PRD.md	Detailed walkthrough of every business-side screen
React-Project-Structure-PRD.md	Frontend architecture and folder conventions
API-Documentation-PRD.md	How the frontend consumes the API

In OHY-SUPER-ADMIN/

File	Contents
OHY Project Requirement-Understanding - PRD.md	Detailed admin requirements

File	Contents
OHY API Documentation - PRD.md	Admin API reference
OHY Cursor Rules - PRD.md	Coding conventions

Live API reference: with the backend running, open <http://localhost:8000/api/documentation> for a browsable, testable list of every endpoint.

A note on precedence: these PRDs describe the system as it was designed. Where a PRD and the actual code differ, the code is the truth — several features (Stripe payments, platform fees, settlements, OTP verification, the CMS, reports and feedback) were added after the original PRDs were written.

13. Glossary

Term	Meaning
API	The backend service the three portals talk to. It has no screens of its own.
Endpoint	A single API address that performs one action, e.g. /api/v1/get_events_list.
Token	A long random string that proves who a logged-in user is. Sent with every request.
Migration	A script that creates or changes a database table. Running migrations builds the database.
Seeder	A script that inserts starter or demo data.
Middleware	A checkpoint that runs before an endpoint and can block the request — used here for permission checks.
Controller	The backend file containing the logic for a group of related endpoints.
Model	A PHP class representing one database table.
Webhook	A message sent from an external service to our backend, e.g. Stripe telling us a payment succeeded.
Stripe Connect / Express account	The mechanism that lets a host receive money directly into their own bank account.
Platform fee	The commission OHY keeps from each sale.
Settlement / Payout	Releasing a host's earnings to their bank account.
CMS	Content Management System — here, the admin's ability to edit static website pages.
OTP	One-Time Password — the 6-digit code emailed to verify an email address.
SPA	Single Page Application — a website that loads once and then swaps content without full page reloads. All three portals are SPAs.
Draft event	An event saved but not yet published; visible only to its host.
Featured event	An event the Super Admin has promoted to the customer homepage.

14. Closing Notes

What you now have

Four applications that together make a complete event ticketing marketplace:

- a **Backend API** holding all data and all business rules,
- a **Business Portal** where organisers create events and track earnings,
- a **Customer Website** where the public discovers events and buys tickets,
- a **Super Admin Portal** where the OHY team governs the platform and releases payouts.

The five things worth remembering

1. **All logic lives in the backend.** The React apps only display screens. To change a rule, change it in the API.
2. **routes/api.php is the map.** Every endpoint, grouped by who can call it. Read it first.
3. **App.tsx is the map for each frontend.** Every screen URL is listed there.
4. **Registration and login happen on the Customer Website only.** Hosts get in by switching profile mode.
5. **php artisan storage:link must be run.** Forgetting it breaks every image on the platform.

Suggested first week for a new developer

Day	Focus
1	Install everything and get all four applications running locally. Complete the five-point checklist in section 9.4.
2	Read routes/api.php end to end. Open /api/documentation and try a few endpoints.
3	Follow one complete flow through the code: customer adds to cart → checkout → Stripe → webhook → order created. This touches most of the system.
4	Create an event through the 7-step wizard while reading the matching controller code in Host/EventController.php.
5	Explore the Super Admin portal: fees, settlements, moderation and CMS.

Recommended immediate actions on taking over

- Change the Super Admin password on every environment.

- Rotate the Stripe, Brevo and Google Maps keys, and remove the hardcoded fallback Maps key from OHY-BUSINESS/src/config/googleMaps.ts.
 - Confirm database backups are scheduled.
 - Update .env.example in the backend to include the Stripe, Brevo and frontend-URL entries.
 - Decide whether to enable the four hidden Business Portal screens (Advanced Analytics, Marketing, Platform Fee, Subscriptions).
 - Set up uptime monitoring against /api/v1/health.
-

Prepared by: Aalpha Information System

For: OHY

End of document.